# A3-PS027-TC404

IPv6 Website and Dependency Readiness Auditor: Browser-Based Failure-Stage Analysis

**NIWISHA S WINSTON & ANGEL M**

niwishas@karunya.edu.in & angelm@karunya.edu.in

AIORI Hackathon 3

October 13, 2026

# Problem Description

- An AAAA record on a website's apex domain does not make the site IPv6-ready.
- Modern pages load scripts, stylesheets, fonts, images and API calls from many third-party hosts, and any of these may be IPv4-only.
- One IPv4-only dependency can break or slow the page for IPv6-only users.
- DNS-only checks miss failures at the TCP, TLS and HTTP stages.
- Existing checkers often report only whether the main domain resolves, not where resource loading fails.

# Background

- IPv6 adoption is growing, and IPv6-only and IPv6-mostly networks (using NAT64/DNS64) are increasingly common, especially on mobile networks.
- A web page is a tree of resources. The HTML references scripts, which load further scripts, fonts and API endpoints, often on different hosts.
- A resource load has four stages: DNS resolution, TCP handshake, TLS negotiation and HTTP retrieval. Each can fail independently, and each needs a different fix.
- Browsers use Happy Eyeballs (fallback to IPv4) on dual-stack networks, which hides IPv6 problems. Testing in an IPv6-only environment exposes them.
- The IETF-inspired webres6 project (SAP) is a reference point for browser-based IPv6 resource testing [1].
- Measurement context matters: results depend on vantage point, resolver and time, so we record them for every run.

# Literature & RFC Review

- **RFC 8200** defines the IPv6 specification and is the baseline for IPv6 behaviour [2].
- **RFC 8305 (Happy Eyeballs v2)** explains why dual-stack clients can mask IPv6 failures through fallback, which is why we test each family in isolation [3].
- **RFC 6146 and RFC 6147 (NAT64/DNS64)** describe how IPv6-only clients reach IPv4-only servers, and why IPv4 literals in pages remain a risk [4][5].
- **RFC 6540 (BCP 177)** requires IPv6 support in IP-capable nodes, which is the motivation for readiness auditing [6].
- **SAP webres6** is an existing browser-based tool for IPv6 resource testing. We build on its approach and add failure-stage classification, confirmation probes and a controlled seeded test site [1].
- **Gap:** DNS-only checkers cannot see failures that happen during real resource loading.

# Proposed Solution

- **Browser automation:** Playwright with headless Chromium, run once per IP family.
- **Resource capture:** Chrome DevTools Protocol events (requestWillBeSent, responseReceived, loadingFailed) give the URL, type, initiator, remote IP and timings.
- **Dependency graph:** the initiator field links each resource to its parent, giving a full tree with depth.
- **Failure-stage classification:** each failed resource is labelled DNS, TCP, TLS or HTTP from error codes and timing data.
- **Confirmation probes:** for each failure, an out-of-browser check runs dig A/AAAA, a TCP connect on port 443 and a TLS handshake.
- **Network control:** IPv4-only and IPv6-only runs are enforced at the container or VM level, and available IP families are checked before every run.

- **Measurement context:** vantage-point IP, country, city and ASN, resolver IP, timestamp and Chromium version.
- **Outputs:** CSV and JSON (one row per resource), an interactive D3 or vis.js graph, and a per-site summary.
- **Validation:** a controlled dual-stack site with seeded failures, plus 30 real websites.

# Optimization Proposed

| Aspect | Before (DNS/ping check) | After (our auditor) |
|---|---|---|
| **Method** | AAAA lookup, or a single request | Real browser resource loading |
| **Scope** | Main domain only | Full dependency tree |
| **Failure detail** | Pass/fail | DNS, TCP, TLS or HTTP stage |
| **Verification** | None | Confirmation probes (dig, TCP, TLS) |
| **Comparison** | Single run | Paired IPv4 and IPv6 runs with a v4-works/v6-fails list |
| **Context** | Not recorded | Geolocation, resolver, timestamp, browser version |
| **Reporting** | Text output | CSV, JSON and interactive graph |
| **Accuracy check** | Unvalidated | Seeded test site with a detection table (e.g. [7/7]) |

# Solution Architecture & Design

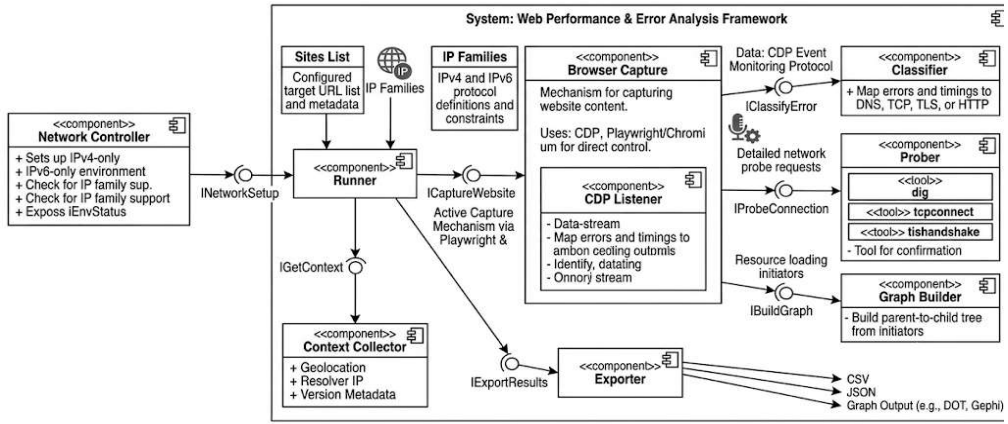



Figure: System Architecture UML Diagram

# Data Flow Diagram

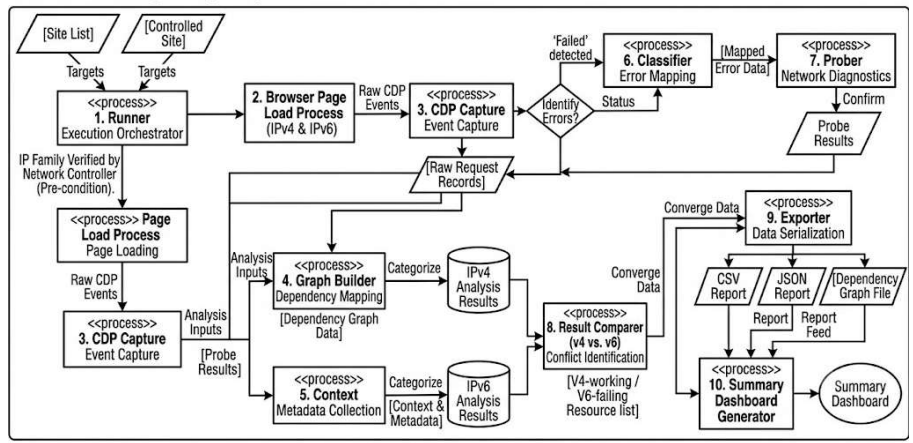



Figure: Data Flow Diagram (DFD)

# Deployment Timeline

Milestones and methodology (adjust dates to your hackathon schedule):

| Phase | Milestone |
|---|---|
| **1** | Set up the repo and the IPv4/IPv6 test environments (container or VM) |
| **2** | Single-page capture: Playwright and CDP, recording requests |
| **3** | Failure classifier and confirmation probes |
| **4** | Dependency graph and CSV/JSON export |
| **5** | Deploy the controlled test site with seeded failures, and validate detection |
| **6** | Run the 30 selected sites on both IP families |
| **7** | Analysis, technical report, intro video and final demo |

**Methodology:** Iterative build, validating each stage on the controlled site before scaling to the 30 sites.

# Bibliography I

[1] SAP. “webres6: Web Resource IPv6 Readiness.” GitHub repository. https://github.com/SAP/webres6
[2] S. Deering and R. Hinden. “Internet Protocol, Version 6 (IPv6) Specification.” RFC 8200, IETF, July 2017.
[3] D. Schinazi and T. Pauly. “Happy Eyeballs Version 2: Better Connectivity Using Concurrency.” RFC 8305, IETF, December 2017.
[4] M. Bagnulo et al. “Stateful NAT64: Network Address and Protocol Translation from IPv6 Clients to IPv4 Servers.” RFC 6146, IETF, April 2011.
[5] M. Bagnulo et al. “DNS64: DNS Extensions for Network Address Translation from IPv6 Clients to IPv4 Servers.” RFC 6147, IETF, April 2011.
[6] D. Brzozowski and C. Byrne. “IPv6 Support Required for All IP-Capable Nodes.” RFC 6540 (BCP 177), IETF, April 2012.

Thank You for Your Attention!